A Technical Introduction to K-nearest neighbors algorithm

Section 1

w n i ∗ = 0 {\displaystyle w_{ni}^{*}=0} for i = k ∗ + 1 , … , n {\displaystyle i=k^{*}+1,\dots ,n} . With optimal weights the dominant term in the asymptotic expansion of the excess risk is O (n − 4 d + 4) {\displaystyle {\mathcal {O}}(n^{-{\frac {4}{d+4}}})} . Similar results are true when using a bagged nearest neighbour classifier.

Section 2

Furthest-neighbor variants A variation of the nearest neighbor approach uses the k furthest neighbors instead of the nearest ones. In this setting, neighbors are selected based on maximal dissimilarity rather than similarity. This approach has been proposed in recommender systems as an "inverted neighborhood" model, where the most dissimilar users are identified and their preferences are used inversely to generate recommendations. The goal is typically to increase diversity or novelty in recommended items, since traditional nearest-neighbor methods may favor popular or obvious items. Empirical evaluations have found that furthest-neighbor approaches generally achieve lower predictive accuracy than standard k-nearest neighbor methods, although user-perceived usefulness may be similar or higher in some cases.

Section 3

Parameter selection The best choice of k depends upon the data; generally, larger values of k reduces effect of the noise on the classification, but make boundaries between classes less distinct. A good k can be selected by various heuristic techniques (see hyperparameter optimization). The special case where the class is predicted to be the class of the closest training sample (i.e. when k = 1) is called the nearest neighbor algorithm. The accuracy of the k-NN algorithm can be severely degraded by the presence of noisy or irrelevant features, or if the feature scales are not consistent with their importance. Much research effort has been put into selecting or scaling features to improve classification. A particularly popular approach is the use of evolutionary algorithms to optimize feature scaling. Another popular approach is to scale features by the mutual information of the training data with the training classes. In binary (two class) classification problems, it is helpful to choose k to be an odd number as this avoids tied votes. One popular way of choosing the empirically optimal k in this setting is via bootstrap method.

Section 4

Properties k-NN is a special case of a variable-bandwidth, kernel density "balloon" estimator with a uniform kernel. The naive version of the algorithm is easy to implement by computing the distances from the test example to all stored examples, but it is computationally intensive for large training sets. Using an approximate nearest neighbor search algorithm makes k-NN computationally tractable even for large data sets. Many nearest neighbor search algorithms have been proposed over the years; these generally seek to reduce the number of distance evaluations actually performed. k-NN has some strong consistency results. As the amount of data approaches infinity, the two-class k-NN algorithm is guaranteed to yield an error rate no worse than twice the Bayes error rate (the minimum achievable error rate given the distribution of the data). Various improvements to the k-NN speed are possible by using proximity graphs. For multi-class k-NN classification, Cover and Hart (1967) prove an upper bound error rate of

Section 5

where R ∗ {\displaystyle R^{*}} is the Bayes error rate (which is the minimal error rate possible), R k N N {\displaystyle R_{kNN}} is the asymptotic k-NN error rate, and M is the number of classes in the problem. This bound is tight in the sense that both the lower and upper bounds are achievable by some distribution. For M = 2 {\displaystyle M=2} and as the Bayesian error rate R ∗ {\displaystyle R^{*}} approaches zero, this limit reduces to "not more than twice the Bayesian error rate".

Section 6

A drawback of the basic "majority voting" classification occurs when the class distribution is skewed. That is, examples of a more frequent class tend to dominate the prediction of the new example, because they tend to be common among the k nearest neighbors due to their large number. One way to overcome this problem is to weight the classification, taking into account the distance from the test point to each of its k nearest neighbors. The class (or value, in regression problems) of each of the k nearest points is multiplied by a weight proportional to the inverse of the distance from that point to the test point. Another way to overcome skew is by abstraction in data representation. For example, in a self-organizing map (SOM), each node is a representative (a center) of a cluster of similar points, regardless of their density in the original training data. k-NN can then be applied to the SOM.

Section 7

The training examples are vectors in a multidimensional feature space, each with a class label. The training phase of the algorithm consists only of storing the feature vectors and class labels of the training samples. In the classification phase, k is a user-defined constant, and an unlabeled vector (a query or test point) is classified by assigning the label which is most frequent among the k training samples nearest to that query point.

Section 8

The 1-nearest neighbor classifier The most intuitive nearest neighbour type classifier is the one nearest neighbour classifier that assigns a point x to the class of its closest neighbour in the feature space, that is C n 1 n n (x) = Y (1) {\displaystyle C_{n}^{1nn}(x)=Y_{(1)}} . As the size of training data set approaches infinity, the one nearest neighbour classifier guarantees an error rate of no worse than twice the Bayes error rate (the minimum achievable error rate given the distribution of the data).

Section 9

Statistical setting Suppose we have pairs (X 1 , Y 1) , (X 2 , Y 2) , … , (X n , Y n) {\displaystyle (X_{1},Y_{1}),(X_{2},Y_{2}),\dots ,(X_{n},Y_{n})} taking values in R d × { 1 , 2 } {\displaystyle \mathbb {R} ^{d}\times \{1,2\}} , where Y is the class label of X, so that X | Y = r ∼ P r {\displaystyle X|Y=r\sim P_{r}} for r = 1 , 2 {\displaystyle r=1,2} (and probability distributions P r {\displaystyle P_{r}}). Given some norm ■ ⋅ ■ {\displaystyle \|\cdot \|} on R d {\displaystyle \mathbb {R} ^{d}} and a point x ∈ R d {\displaystyle x\in \mathbb {R} ^{d}} , let (X (1) , Y (1)) , … , (X (n) , Y (n)) {\displaystyle (X_{(1)},Y_{(1)}),\dots ,(X_{(n)},Y_{(n)})} be a reordering of the training data such that ■ X (1) − x ■ ≤ ■ ≤ ■ X (n) − x ■ {\displaystyle \|X_{(1)}-x\|\leq \dots \leq \|X_{(n)}-x\|} .